When Thermal-Margin Control Helps and When It Hurts: A Matched-Baseline Study of Convex Allocation for Multi-Tenant Edge Inference

Manu Nicholas Jacob

***Abstract*—Convex *thermal-margin* allocation treats the gap to an operator temperature cap as a resource, priced across tenants by a dual variable. We ask whether the tenant-level machinery earns its complexity, using the experiment that isolates it: on a Raspberry Pi 5 serving three single-threaded ONNX inference tenants under open-loop arrivals, we compare the convex allocator against equal sharing, a static quota, a *utilization-matched* baseline that replays the allocator's own aggregate duty trace as an equal split, and a two-line *admission* rule that inverts a utilization–temperature law to the same aggregate budget. The matched baseline reproduces the convex allocator's goodput, violations, and tail latency (goodput ratio 1.01), and the admission rule is equivalent: with near-linear tenant rate curves, proportional fairness reduces to an equal budget split, so *everything the controller does is carried by the aggregate cap*. The cap buys temperature (59.9 vs. 55.6°C at high load); its cost under open-loop arrivals is bistable: at medium load, $\sim$2 °C of ambient drift decides whether the cap binds, and when it does, duty throttling converts thermal margin into queue collapse (p99 15.6 s against 841 ms for equal sharing, unaffected in every run) while a slack cap changes nothing. An earlier closed-loop evaluation that measured service time without queueing delay reported the opposite tail result; the correction is part of the contribution. Practitioner rule: hold a thermal cap by shedding requests, not throttling duty cycles; if duty throttling is used, an aggregate budget is all that is needed.**

***Index Terms*—Edge inference, thermal management, resource allocation, matched baselines, tail latency, multi-tenancy.**

I. Introduction

MULTI-TENANT inference on thermally constrained single-board computers invites predictive resource allocation: model the platform's thermal response, treat the margin to a temperature cap as an allocatable resource, and split it across tenants by convex optimization [1]. The construction is appealing, and variants of it appear across datacenter QoS [2], [3] and edge serving [4]. This letter asks the question such proposals rarely isolate: *which part of the machinery does the work?* A tenant-level convex program embeds two separable decisions: how much aggregate capacity the thermal cap permits, and how that aggregate is divided among tenants. Only the second is "convex allocation"; the first is a scalar budget any admission rule can compute.

We isolate the two with matched baselines on real hardware. Three single-threaded ONNX tenants (MobileNetV2, ResNet-18, SqueezeNet [5]) run on a Raspberry Pi 5 under open-loop Poisson, bursty, and saturating arrivals, with duty cycles enforced by cgroup CPU quotas. The convex allocator maximizes proportional-fair log-throughput subject to a temperature cap through a calibrated utilization–temperature law with integral feedback. Against it we run: **equal** (conventional OS scheduling, no caps); **quota** (static per-tenant cap); **admission**, which computes the *same* law-plus-feedback aggregate budget and splits it equally, with no solver, no rate curves, and no per-tenant reasoning; and **matched**, which replays the convex run's own per-second aggregate duty trace as an equal split. If tenant-level allocation matters, convex must beat matched; if the budget computation matters, admission must trail convex. Neither happens.

Findings, across two tenant mixes, four load levels, three arrival processes, and three cap levels:

- **The aggregate cap carries everything.** Where the cap binds, matched reproduces convex within run-to-run noise on goodput, violations, and p99 (goodput ratio 1.01); admission is likewise indistinguishable. With measured near-linear tenant rate curves, proportional fairness collapses to an equal budget split, and the solver prices nothing the budget did not already decide (§III).
- **Under open-loop load, duty throttling buys temperature with a queueing cliff.** The cap holds the SoC 4.2°C cooler at high load, but the throttled service rate sits near the queueing stability boundary: at medium load, $\sim$2 °C of ambient drift decides between no effect at all (cap slack, capped policies $\equiv$ equal sharing) and total collapse (cap binding: goodput 2.0 vs. 25 req/s, p99 15.6 s vs. 841 ms) (§IV).
- **A closed-loop evaluation inverts the tail conclusion.** Measuring per-request service time without arrival queueing, as our own earlier evaluation did, reports throttling as a tail-latency *win*; with latency measured from arrival, the sign reverses. We document the pitfall so it is not repeated (§IV).

Everything (harness, controller, raw per-request and per-second logs, analysis) is released.

M. N. Jacob received the dual B.S. degree in Electrical Engineering and Computer Engineering from the University of Massachusetts Amherst, Amherst, MA, USA, in 2025 (e-mail: manunicholasjacob@gmail.com; ORCID: 0009-0007-6589-6572).

Code and all telemetry are released at https://github.com/manunicholasjacob/edge-thermal-margin-control.

II. System and Policies

Platform and tenants. Raspberry Pi 5 (quad-core Cortex-A76, 2.4 GHz pinned, active cooling), ONNX Runtime, one thread per tenant. Tenants serve a bounded FIFO queue fed by an arrival process (Poisson at a per-tenant rate, an on–off burst process, or closed-loop saturation); *latency is measured from arrival to completion*, so queueing is included, and queue overflows count as violations. Two mixes: homogeneous (3×MobileNetV2) and heterogeneous (MobileNetV2, ResNet-18, SqueezeNet; solo rates 29, 7.8, 34 req/s). Loads are per-tenant fractions of solo rate: 0.3 (low), 0.5 (medium), 0.7 (high), 1.0 (extreme).

Thermal model. A calibration measured at campaign start refits the platform's utilization–temperature law $T_{ss} = b_0 + k\,U$ (idle intercept 45.8°C, slope 0.183°C/% at this ambient and cooling; the slope agrees within 5% with a 13-hour characterization taken six months earlier). Caps are placed inside the measured thermal band [45.8, 59.5]°C at 35% (tight, 50.6°C), 60% (primary, 54.0°C), and 85% (loose, 57.4°C) of the idle-to-soak range.

Policies. Duties $x_i \in [0.05, 1]$ core-units are enforced per tenant as cgroup-v2 `cpu.max` quotas at a 1 Hz control rate. The shared budget logic seeds the aggregate $X = \sum_i x_i$ from the inverted law at the cap and trims it each tick by integral feedback on the measured temperature error, identically for convex and admission, so the two differ *only* in how X is split: *convex* solves $\max \sum_i \log \mu_i(x_i)$ s.t. $\sum_i x_i \le X$ with per-tenant rate curves μ_i measured at four duty levels; *admission* sets $x_i = X/n$; *matched* replays a completed convex run's per-second $X(t)$ as $x_i = X(t)/n$; *quota* fixes $x_i = 0.6$; *equal* applies no caps, which for single-threaded tenants on four cores is conventional fair scheduling.

III. Where the Machinery Does and Does Not Matter

Table I and Fig. 1 are the letter's central result. At low load the cap does not bind and all policies coincide. At medium load the campaign surfaced something a mean over replications would hide: the capped system is *bistable*. Overnight ambient drift of ∼2 °C decides whether the cap binds; both states hold the same temperature (mean 53.8 °C controlled); what differs is the duty the feedback needs to hold it. In the slack state (med^s, 5 of the nine capped runs, mean aggregate duty 2.9 cores) every capped policy matches equal sharing; in the binding state (med^b, 4 runs, 1.6 cores) every capped policy collapses together while equal sharing is unaffected in all of its runs. Within either state, the comparisons that isolate the machinery are unambiguous: matched reproduces convex, its replay partner, on goodput, violations, and p99, and admission lands in the same state with the same outcomes. The mechanism is visible in the solver's own output: with measured near-linear rate curves, the proportional-fair optimum deviates from an equal split of the budget by 0.4% on average, heterogeneous mix included. The convex program is not wrong; it is *redundant*, at ∼25 ms of solver per tick against microseconds for the division X/n, and its allocation decisions never separate its outcomes from an equal split of the same budget.

TABLE I
Matched-baseline comparison, heterogeneous mix, Poisson arrivals, primary cap (tight SLA of 2.5× solo median service latency; "gp" = SLA-met goodput, req/s). Medium load is split by the realized thermal state of the capped runs: med^s = cap slack, med^b = cap binding; equal sharing has no cap and is listed once per load.

Load	Policy	gp	viol %	p99 (ms)	$T_{\max}$ (°C)
low	equal	20.4	7	417 ms	50.7
	convex	20.4	6	418 ms	51.8
	matched	20.5	6	421 ms	50.3
	admission	20.5	6	418 ms	50.7
med^s	equal	24.6	31	841 ms	56.6
	convex	23.7	34	851 ms	55.6
	matched	23.5	35	858 ms	56.5
	admission	23.8	35	974 ms	56.2
med^b	convex	0.1	99	15.5 s	55.6
	matched	0.0	100	22.5 s	54.0
	admission	3.9	88	12.3 s	55.4
high	equal	0.6	98	9.9 s	59.9
	convex	0.0	100	66 s	55.6
	matched	0.0	100	62.7 s	54.4
	admission	0.0	100	61.2 s	55.5
extreme	equal	0.0	100	57.1 s	59.9
	convex	0.0	100	94.4 s	55.3
	matched	0.0	100	97.8 s	56.0
	admission	0.0	100	95.9 s	55.1

IV. The Price of Duty Throttling Under Open-Loop Load

The second finding concerns what thermal duty-throttling costs when requests keep arriving. Equal sharing runs hotter, up to 59.9°C at high load against 55.6°C for the capped policies, and the capped policies exceed the cap by more than 0.5 °C (the sensor's granularity) in only 19% of controlled seconds, against 80% for equal sharing under binding load. But under open-loop arrivals the removed CPU time does not disappear; it becomes queue depth, and it does so as a cliff rather than a slope. At medium load the throttled service rate sits near the queueing stability boundary, so whether the system survives is decided by the thermal state: in the slack state the capped policies serve 24 req/s against equal sharing's 25 with comparable tails (878 ms), and in the binding state they serve 2.0 req/s with p99 of 15.6 s while equal sharing still delivers 25 req/s at 841 ms. A ∼2 °C warmer ambient is the difference between no effect and total collapse. At high and extreme load queues grow without bound and every policy saturates its violation rate, capped policies first. There is no regime, at any of the three caps or under any arrival process, in which throttling improves the tail: the tight cap strangles service (p99 in the tens of seconds at medium load), the loose cap approximates no control, and under bursty on–off arrivals the idle periods keep the cap from binding at all, making every policy identical. Even under closed-loop saturation, where queueing cannot be measured by construction, quota throttling stretches individual service times (worst-tenant p99 336 ms against 190 ms unthrottled). The pattern is not particular to one configuration. The homogeneous mix reproduces it: at medium load, equal sharing serves 21.3 req/s, capped policies serve 20.6 in the slack state and 0.2 (p99 21 s) in the two runs where the cap bound. Relaxing the SLA to 6× the

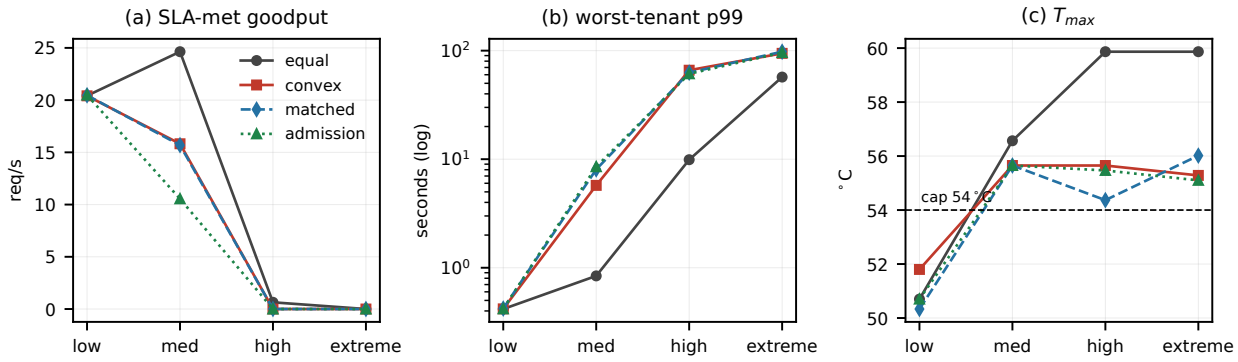

Fig. 1. Load sweep, heterogeneous mix, Poisson arrivals, primary cap (mean over reps; the medium-load points for capped policies average the two thermal states that Table I separates). (a) Where the cap binds, convex, matched, and admission coincide; equal sharing dominates goodput. (b) Capped policies pay for temperature in queueing: worst-tenant p99, log scale. (c) What the cap buys: $T_{\max}$ held near the cap while equal sharing runs to the top of the thermal band.

solo median changes the numbers, not the ordering: 35.2 req/s for equal sharing against 34.7 slack and 3.5 binding. At the tight cap both thermal policies collapse together (goodput 0.0, p99 46–58 s); at the loose cap both serve 24 req/s at 0.9 s, indistinguishable from equal sharing.

This inverts the conclusion of our own earlier closed-loop evaluation, which measured per-request *service* time under saturating tenants and reported zero violations and a 3.7× p99 reduction for the convex controller. That number answered the wrong question: with no arrival process, waiting time does not exist by construction. Under the corrected harness the earlier win does not reappear under either protocol: throttling produces the worst tails in the study when latency is measured from arrival, and it still lengthens service tails under closed-loop saturation. Evaluations of throttling controllers that report service latency alone will systematically overstate them.

Practitioner rule. Hold a thermal cap by *shedding or deferring requests* (admission control at the request level), which keeps served-request latency intact, rather than by duty-throttling the service rate under unshed load. If duty throttling is used, compute the aggregate budget from an inverted thermal law with feedback; per-tenant convex machinery adds solver cost and nothing else at this workload class.

V. Related Work

Datacenter QoS managers [2], [3] and inference serving systems [4] operate at request level, where load that cannot be served is shed or deferred before it queues, which is exactly the property duty throttling lacks; thermal actuation on mobile SoCs is dominated by DVFS, including learning-based control such as zTT [6], which actuates frequency rather than tenant duty cycles and targets frame rate. Temperature-aware scheduling has deep processor-level roots [7], [8]. Our contribution is not a new controller but a matched-baseline accounting of an appealing one: at application level on a stock SBC, with the frequency governor pinned, the thermally relevant decision is a scalar budget, and the tail cost of enforcing it by duty throttling under open-loop load has not, to our knowledge, been isolated before.

VI. Limitations

One board, one cooling configuration, and a thermal band of ∼14°C; caps are placed within it, and colder or passive setups will move the numbers though not the mechanism. Tenants are single-threaded ONNX CPU models; multi-threaded tenants couple through the scheduler and may reward tenant-level allocation differently. The convex allocator's redundancy is a property of near-linear measured rate curves and symmetric log-utility weights; strongly concave per-tenant utilities or priority weights could re-differentiate it, and our claim is scoped to the common proportional-fair configuration. Runs are 180 s per cell with 2–3 replications, letter-scale rather than exhaustive.

VII. Conclusion

With the baseline the question deserves, a utilization-matched equal split of its own duty trace, convex thermal-margin allocation shows no tenant-level benefit on a Raspberry Pi 5 serving three real inference tenants: the aggregate budget carries every effect, a two-line admission rule reproduces it, and under open-loop arrivals duty throttling itself trades temperature for order-of-magnitude tail inflation. The honest deployment guidance is to enforce thermal caps at the request level and, where duty throttling is unavoidable, to skip the per-tenant solver. We release the full campaign so the matched-baseline protocol can be reused.

Acknowledgment

No external funding was received for this work. The experimental design, measurements, analyses, and conclusions are the author's own. Generative AI (Claude, Anthropic) assisted with manuscript editing, LaTeX, and some implementation of supporting experimentation. All of its output was reviewed and verified by the author.

References

[1] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge Univ. Press, 2004.
[2] C. Delimitrou and C. Kozyrakis, "Quasar: Resource-Efficient and QoS-Aware Cluster Management," in *Proc. ASPLOS*, 2014.

[3] D. Lo *et al.*, "Heracles: Improving Resource Efficiency at Scale," in *Proc. ISCA*, 2015.
[4] D. Crankshaw *et al.*, "Clipper: A Low-Latency Online Prediction Serving System," in *Proc. NSDI*, 2017.
[5] M. Sandler *et al.*, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," in *Proc. CVPR*, 2018.
[6] S. Kim *et al.*, "zTT: Learning-Based DVFS with Zero Thermal Throttling for Mobile Devices," in *Proc. ACM MobiSys*, 2021, pp. 41–53.
[7] K. Skadron *et al.*, "Temperature-Aware Microarchitecture," in *Proc. ISCA*, 2003, pp. 2–13.
[8] A. K. Coskun, T. S. Rosing, and K. C. Gross, "Proactive Temperature Management in MPSoCs," in *Proc. ACM/IEEE ISLPED*, 2008, pp. 165–170.